Loss Functions in Neural Networks

A loss function tells a neural network how wrong its prediction is compared with the actual target.

During training, the neural network tries to minimize the loss by adjusting its weights and biases using an optimizer such as SGD or Adam.

A simple flow is:

```
Input
  |
  v
Neural Network
  |
  v
Prediction (y_hat)
  |
  v
Loss Function
  |
  v
Loss value
  |
  v
Backpropagation
  |
  v
Gradients
  |
  v
Optimizer updates weights
```

The most important point is: different problems require different loss functions because “error” means different things for regression, binary classification, and multiclass classification.

MSE — Mean Squared Error

What is MSE?

Mean Squared Error (MSE) is one of the most commonly used loss functions for regression problems.

Regression means that the output is a continuous numerical value.

Examples:

Problem	Target
House price prediction	₹65,00,000
Temperature prediction	28.5°C
Salary prediction	₹75,000
Taxi fare prediction	₹350
Sales prediction	12,500 units

The model produces a numerical prediction, and MSE measures how far that prediction is from the actual value.

Mathematical expression

$$MSE = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2$$

Where:

- n = number of observations
- y_i = actual value
- $\hat{y}_i$ = predicted value
- $y_i - \hat{y}_i$ = prediction error

The important operation is that the error is squared.

Example

Actual	Predicted
100	90
200	220
300	270

Errors:

$$100 - 90 = 10$$

$$200 - 220 = -20$$

$$300 - 270 = 30$$

Squared errors:

$$10^2 = 100$$

$$(-20)^2 = 400$$

$$30^2 = 900$$

Therefore:

$$MSE = \frac{100 + 400 + 900}{3} = 466.67$$

So the loss is 466.67.

Why does MSE square the error?

This is extremely important. Suppose we have two errors:

$$\text{Error} = 2 \qquad \text{Error} = 10$$

Without squaring, the second error is 5 times larger. After squaring:

$$2^2 = 4 \qquad 10^2 = 100$$

Now the second error contributes 25 times more to the loss.

Therefore: MSE heavily penalizes large errors.

This makes MSE particularly useful when large prediction errors are considered especially undesirable.

Advantages of MSE

1. Penalizes large errors strongly

Because errors are squared:

$$10^2 = 100 \qquad 2^2 = 4$$

Large errors dominate the loss.

2. Smooth and differentiable

MSE has a smooth curve, which makes it convenient for gradient-based optimization.

3. Simple to understand

It directly measures the average squared difference between actual and predicted values.

4. Good default for many regression problems

MSE works particularly well when the noise in the target is approximately Gaussian and large errors should receive strong penalties.

Disadvantages of MSE

The biggest disadvantage is: MSE is sensitive to outliers.

Suppose most errors are 2, 3, 4, 2, 3 but one observation has 100. MSE sees:

$$100^2 = 10{,}000$$

That single observation can dominate the loss.

Use MSE when:

- target is continuous
- large errors should be strongly penalized
- data does not contain severe outliers

MAE — Mean Absolute Error

What is MAE?

Mean Absolute Error (MAE) is another loss function used for regression.

Instead of squaring the error, MAE takes the absolute value.

Mathematical expression

$$MAE = \frac{1}{n}\sum_{i=1}^{n}|y_i - \hat{y}_i|$$

The key difference is:

$$\text{MSE}: \ (y-\hat{y})^2 \qquad\qquad \text{MAE}: \ |y-\hat{y}|$$

Example of MAE

Using the same example:

Actual	Predicted	Error	Absolute Error
100	90	10	10
200	220	-20	20
300	270	30	30

Therefore:

$$MAE = \frac{10+20+30}{3} = 20$$

So the MAE is 20.

MSE vs MAE

Consider two errors:

$$\text{MSE: } 2^2 = 4 \qquad 10^2 = 100$$

$$\text{MAE: } |2| = 2 \qquad |10| = 10$$

	Small error	Large error
MSE	4	100
MAE	2	10

MSE gives the large error much more importance. MAE treats errors more linearly.

Why is MAE more robust to outliers?

Suppose an error is 100:

$$\text{MSE: } 100^2 = 10{,}000$$

$$\text{MAE: } |100| = 100$$

Therefore, an extreme error has a much smaller influence under MAE than under MSE.

So: MAE is generally more robust to outliers than MSE.

Advantages of MAE

- Less sensitive to outliers
- Easy to interpret
- Error is in the same units as the target

- Useful when all errors should be treated approximately proportionally

For example, if predicting house prices, MAE can be interpreted as: "On average, the prediction is off by ₹X."

Disadvantages of MAE

MAE has an important mathematical characteristic. The absolute value function |x| has a sharp point at x = 0. It is therefore not differentiable exactly at zero.

Modern optimizers can still work with MAE, but its optimization behavior can be different from the smooth MSE objective.

Another limitation is that MAE does not strongly prioritize very large errors.

MSE vs MAE — When should you use which?

Characteristic	MSE	MAE
Error calculation	Squared	Absolute
Large errors	Strongly penalized	Less strongly penalized
Outlier sensitivity	High	Lower
Smoothness	Smooth	Not differentiable at 0
Interpretation	Squared target units	Same units as target
Use with outliers	Usually less suitable	Usually better
Common use	Standard regression	Robust regression

Simple rule:

No significant outliers + large errors are costly → MSE

Outliers present + want robustness → MAE

Binary Cross-Entropy

Now we move from regression to classification.

Binary classification means there are two possible classes. Examples:

- Spam / Not Spam
- Fraud / Not Fraud
- Disease / No Disease

- Pass / Fail
- Cat / Not Cat

Usually we represent the target as 0 or 1.

Why don't we use MSE for classification?

Suppose a model predicts probability = 0.9 for class 1. If the actual class is y = 1, that's a good prediction.

But classification is fundamentally about probabilities and likelihood, not simply numerical distance.

Binary cross-entropy is designed specifically for this situation.

Binary Cross-Entropy — Mathematical expression

For one observation:

$$L = -[\, y\log(p) + (1-y)\log(1-p)\,]$$

Where:

- y = actual class, either 0 or 1
- p = predicted probability of class 1

For n observations:

$$BCE = -\frac{1}{n}\sum_{i=1}^{n}[y_i\log(p_i) + (1-y_i)\log(1-p_i)]$$

Understanding BCE intuitively

Suppose the actual class is y = 1. Then 1 − y = 0, so the equation becomes:

$$y=1 \Rightarrow 1-y=0 \Rightarrow L=-\log(p)$$

Now consider different predictions.

Actual	Predicted probability	Loss
1	0.99	Very small
1	0.90	Small
1	0.60	Moderate
1	0.10	Very large
1	0.01	Extremely large

This demonstrates something very important:

BCE heavily penalizes confident wrong predictions.

Why does BCE punish confident wrong predictions?

Suppose actual y = 1 but model predicts p = 0.01. Then:

$$y=1,\ p=0.01: \quad L=-\log(0.01)\approx 4.605$$

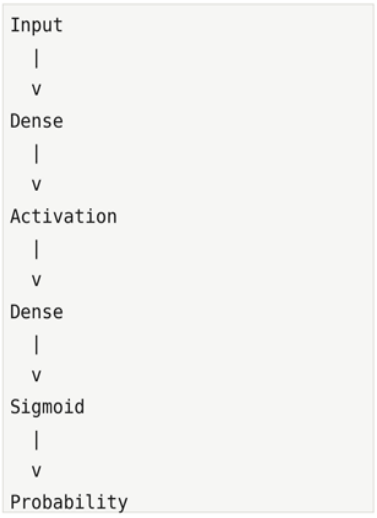
$$y=1,\ p=0.99: \quad L=-\log(0.99)\approx 0.010$$

So: Prediction = 0.99 → tiny loss. Prediction = 0.01 → huge loss.

This is exactly what we want from a classification loss.

Binary classification architecture

A typical neural network looks like:



The final layer might be: Dense(1, activation="sigmoid")

Sigmoid produces a value between 0 and 1. For example:

Sigmoid output	Likely class
0.02	probably class 0
0.25	probably class 0
0.51	probably class 1
0.95	probably class 1

Then BCE evaluates whether that probability agrees with the actual class.

This is appropriate when:

- Number of classes = 2
- Target = 0 or 1
- Output = sigmoid probability
- Loss = binary cross-entropy

Categorical Cross-Entropy

Now consider multiclass classification. Suppose we want to classify handwritten digits 0–9. There are K = 10 classes.

Unlike binary classification, we now need a probability for each class. For example:

Class	Probability
0	0.01
1	0.02
2	0.01
3	0.03
4	0.04
5	0.02
6	0.01
7	0.82
8	0.02
9	0.02

The probabilities should sum approximately to 1.

Softmax

The final layer normally uses softmax:

Dense(10, activation="softmax")

Softmax converts the raw outputs into probabilities. For example:

```
Raw outputs
    |
    v
  Softmax
    |
    v
[0.01, 0.02, 0.01, 0.03, 0.04,
 0.02, 0.01, 0.82, 0.02, 0.02]
```

The largest probability is for class 7. Therefore: Prediction = 7.

Categorical Cross-Entropy

The mathematical expression is:

$$L = -\sum_{k=1}^{K} y_k \log(p_k)$$

Where:

- K = number of classes
- y□ = actual value for class k
- p□ = predicted probability for class k

One-hot encoding

Categorical cross-entropy normally expects one-hot encoded labels. For 10 classes, actual class = 7 becomes:

```
Class:      0 1 2 3 4 5 6 7 8 9

Target:     0 0 0 0 0 0 0 1 0 0
```

Only the correct class has a value of 1.

How does CCE calculate the loss?

Suppose the actual class is 7 and the model predicts a probability of 0.82 for class 7.

Because all y□ values are zero except class 7, only class 7 contributes:

$$L = -\log(0.82)$$

Therefore, the loss is relatively small.

What if the model is confidently wrong?

Suppose the actual class is 7 but the model predicts class 7 probability = 0.01. Then:

$$L = -\log(0.01)$$

Therefore: categorical cross-entropy strongly penalizes the model when it assigns a low probability to the correct class.

Sparse Categorical Cross-Entropy

This is where many beginners get confused.

Categorical cross-entropy and sparse categorical cross-entropy use essentially the same underlying cross-entropy idea.

The major difference is how the target labels are represented.

Categorical Cross-Entropy

Uses one-hot labels. Example: actual class = 7 → [0,0,0,0,0,0,0,1,0,0]

Sparse Categorical Cross-Entropy

Uses the integer class label directly. Example: actual class = 7 → 7

Example with MNIST

Suppose y_train[0] = 7. This is an integer label. Then you can use:

loss="sparse_categorical_crossentropy"

You do not need to convert 7 into [0,0,0,0,0,0,0,1,0,0].

If you convert the labels into one-hot encoding, then use:
loss="categorical_crossentropy"

CCE vs Sparse CCE

Feature	Categorical CE	Sparse Categorical CE
Problem	Multiclass	Multiclass
Classes	2+	2+
Label format	One-hot	Integer
Example	[0,0,1,0]	2
Output	Softmax	Softmax
Underlying idea	Cross-entropy	Cross-entropy
Main difference	Target representation	Target representation

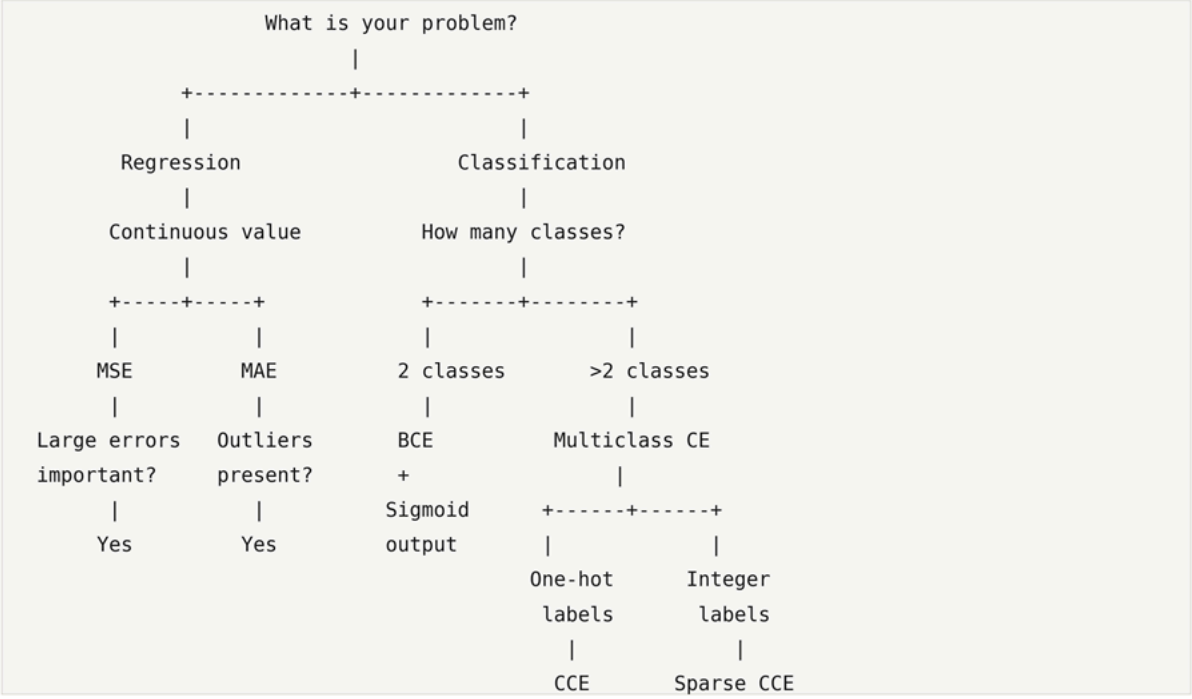
Very important:

Sparse does NOT mean sparse predictions. It means the target labels are represented sparsely as class indices rather than a one-hot vector.

Complete comparison

Problem	Loss	Output layer	Target format
Regression	MSE	Linear	Continuous
Regression	MAE	Linear	Continuous
Binary classification	Binary CE	Sigmoid	0/1
Multiclass	Categorical CE	Softmax	One-hot
Multiclass	Sparse Categorical CE	Softmax	Integer

A very useful decision tree



The most important conceptual difference

Think about what each loss is trying to measure.

MSE

"How far is my numerical prediction from the actual numerical value?"

MAE

"How much numerical error did I make, without excessively emphasizing large errors?"

Binary Cross-Entropy

"How well does my predicted probability match the correct binary class?"

Categorical Cross-Entropy

"How much probability did I assign to the correct class?"

Sparse Categorical Cross-Entropy

"Same multiclass objective, but my correct class is provided as an integer rather than a one-hot vector."